Ada — Reference

An **Ada** subset compiled with `lex` + `yacc` + `cc` (`ada.l` / `ada.y` lower Ada to C; `cc` lowers the C to .NET IL). The main procedure becomes `main`; every other subprogram (including ones nested in the main's declarative part) is hoisted to a `public static` method on `CProgram`, so compiled Ada is callable from C#/VB.NET.

Ada is **strongly, statically typed** and case-insensitive. It was deliberately designed to be LALR(1)-parseable, so it maps cleanly onto our toolchain.

```
ada prog.adb                # compile to prog.exe (native .NET executable)
ada prog.adb -o app.exe     # choose the output name
ada lib.adb --dll           # compile to a library for C#/VB.NET to reference
```

Quick Reference

```
with Ada.Text_IO; use Ada.Text_IO;
procedure Main is
   X : Integer := 5;                -- variable with initializer
   Pi : constant Float := 3.14159;  -- constant
   type Color is (Red, Green, Blue); -- enumeration
   A : array (1 .. 10) of Integer;  -- array, indexed A(i), 1-based
   function Sq (N : Integer) return Integer is        -- function (by value)
   begin return N * N; end Sq;
   procedure Swap (P : in out Integer; Q : in out Integer) is  -- in out = by reference
   begin ... end Swap;
begin
   Put_Line ("text" & Integer'Image (X));   -- & concatenates; 'Image → string
   Put (Integer'Image (X));   New_Line;
   X := X + 1;                A (3) := 9;
   if C then ... elsif D then ... else ... end if;
   case N is when 1 ⇒ ...; when 2 | 3 ⇒ ...; when others ⇒ ...; end case;
   for I in 1 .. 10 loop ... end loop;        for I in reverse 1 .. 10 loop ...
   while Cond loop ... end loop;              loop ... exit when Done; end loop;
end Main;
```

Data types

`Integer` (32-bit), `Float` (double), `Boolean` (`True` / `False`), `Character`, and `String`. **Enumerations** (`type Color is (Red, Green, Blue)`) are first-class — the literals are ordered constants. **Arrays** (`array (Lo .. Hi) of T`, named or anonymous) are indexed with parentheses and are 1-based by their lower bound. Numeric literals, `'c'` character literals, `"..."` strings.

Statements / Commands

Assignment (`:=`), `if/elsif/else/end if`, `case … when … when others … end case`, the loop forms (`loop`, `while … loop`, `for I in [reverse] L .. H loop`) with `exit` / `exit when`, procedure calls, `return`, `null;`, and `Ada.Text_IO` output.

Functions

Two kinds of subprogram. A **function** returns a value (`return expr;`) and a **procedure** returns through its parameters or not at all. Parameter modes: `in` (the default) passes **by value**; `out` and `in out` pass **by reference**, so `Swap (X, Y)` exchanges the caller's variables. Each subprogram compiles to a `public static` method `ada_<name>` on `CProgram` — the basis for C#/VB.NET interop (Activity 9).

Input / Output

Via `Ada.Text_IO`: `Put_Line (S)` writes a string and a newline, `Put (S)` without the newline, `New_Line` ends a line. Numbers are turned into strings with the `'Image` attribute (`Integer'Image (X)`, `Float'Image (X)`) — note Ada's `'Image` puts a leading space before a non-negative number, which this subset reproduces.

Graphics

None. Ada is general-purpose/systems-oriented; Activity 8 below builds a text histogram.

Notes

- **Case-insensitive**: `Put_Line`, `put_line`, and `PUT_LINE` are the same.
- `=` is equality, `/=` is inequality, `:=` is assignment, `&` concatenates strings.
- `and then` / `or else` are the short-circuit forms; `mod` / `rem` / `abs` / `**` are operators.
- The `'` tick is an attribute marker (`X'Image`) except where a character literal (`'a'`) is expected — disambiguated by context in the scanner.

Subset boundaries

A solid procedural core. Not included: packages with separate spec/body (`package … is`), generics, tasking, exceptions, `record` types, access (pointer) types, `declare` blocks, 2-D arrays and array slices/attributes (`'First` / `'Last` / `'Length`), and the wider standard library. Nested subprograms are *hoisted* (they don't capture the enclosing procedure's locals). Numbers are 32-bit `Integer` / 64-bit `Float`.

Tutorial

Every example was compiled and run with `ada`; the output shown is real.

1. Your first program

```
with Ada.Text_IO; use Ada.Text_IO;
procedure Main is
begin
   Put_Line ("Hello from Ada");
end Main;
```

```
Hello from Ada
```

`with` / `use` make `Ada.Text_IO` visible; `procedure Main` is the entry point and becomes `main`.

2. Variables and data types

```
X    : Integer := 5;
Name : String  := "Ada";
Ok   : Boolean := True;
type Color is (Red, Green, Blue);
C    : Color := Green;
```

Each object is declared with a type after `:`. Enumerations like `Color` give named, ordered values (`Green` is the constant 1).

3. Flow control

```
if N > 10 then
   Put_Line ("big");
elsif N > 0 then
   Put_Line ("small");
else
   Put_Line ("zero");
end if;

case C is
   when Red   => Put_Line ("red");
   when Green => Put_Line ("green");
   when Blue  => Put_Line ("blue");
end case;
```

With `N = 5` and `C = Green`:

```
small
green
```

`case … when` is the multi-way branch (`when others =>` is the catch-all); `elsif` chains conditions.

4. Arrays

Arrays are 1-based and indexed with parentheses:

```
A : array (1 .. 5) of Integer;
Sum : Integer := 0;
...
for I in 1 .. 5 loop
   A (I) := I * I;
end loop;
for I in reverse 1 .. 5 loop
   Sum := Sum + A (I);
end loop;
Put_Line ("Squares sum 1..5 =" & Integer'Image (Sum));
```

```
Squares sum 1..5 = 55
```

`for I in reverse 1 .. 5` counts down; `A (I)` indexes the array.

5. Subroutines and functions

A **function** returns a value; a **procedure** with `in out` parameters mutates the caller's variables:

```
function Square (N : Integer) return Integer is
begin
   return N * N;
end Square;

procedure Swap (A : in out Integer; B : in out Integer) is
   T : Integer;
begin
   T := A;  A := B;  B := T;
end Swap;
```

With `X = 5`: `Integer'Image (Square (X))` gives `25`, and after `Swap (X, Total)` the two variables are exchanged — exactly Ada's `in out` by-reference semantics.

6. Memory management

Storage is **static / stack** — there is nothing to allocate or free:

- Each variable is a fixed object for the lifetime of its subprogram (or the program, for the main's variables).
- `array (1 .. N) of T` reserves `N` elements; `constant` makes a read-only value.
- Subprogram parameters and locals live for the duration of the call.

Heap allocation (access types) is outside this subset.

7. Strings and enumerations

Strings are joined with `&`, and `'Image` turns any scalar into its textual form:

```
type Day is (Mon, Tue, Wed);
D : Day := Wed;
N : Integer := 42;
Put_Line ("Day" & Integer'Image (D) & ", N =" & Integer'Image (N));
```

```
Day 2, N = 42
```

`'Image` of an enumeration gives its position (`Wed` = 2); concatenation builds the line.

8. Drawing a picture — a text histogram

```
with Ada.Text_IO; use Ada.Text_IO;
procedure Main is
   Data : array (1 .. 4) of Integer;
   Bar  : String := "";
begin
   Data (1) := 3;  Data (2) := 6;  Data (3) := 2;  Data (4) := 5;
   for I in 1 .. 4 loop
      Bar := "";
      for J in 1 .. Data (I) loop
         Bar := Bar & "#";
      end loop;
      Put_Line (Bar);
   end loop;
end Main;
```

```
###
######
##
#####
```

Each row concatenates `Data (I)` hashes — a histogram drawn with `&` and a nested loop.

9. Calling Ada from C# / VB.NET

Compile an Ada file of subprograms as a library; each becomes a `public static` method on `CProgram`:

```
function Add (A : Integer; B : Integer) return Integer is
begin
   return A + B;
end Add;

function Fib (N : Integer) return Integer is
   A : Integer := 0;  B : Integer := 1;  T : Integer;
begin
   for I in 1 .. N loop
      T := A + B;  A := B;  B := T;
   end loop;
   return A;
end Fib;
```

```
ada lib.adb --dll          # → adalib.dll
```

A C# program referencing `adalib.dll` (and `CRuntime.dll`) calls them directly:

```
Console.WriteLine($"add(20, 22) = {CProgram.ada_add(20, 22)}");
Console.WriteLine($"fib(15)     = {CProgram.ada_fib(15)}");
```

```
add(20, 22) = 42
fib(15)     = 610
```

Because parameters are by value, the signatures are ordinary .NET ones (`int ada_add(int, int)`) — full type checking on the C# side. From here the natural extensions are records, packages, and exceptions (*Subset boundaries*).